Name:Manjyot Wadhwa
Roll no : 65
Div: D15C

EXPERIMENT 4

Implementation of K-Nearest Neighbors (KNN) for Heart Disease Prediction

---

1. Dataset Source

Dataset Name: Heart Disease Dataset
Source: Kaggle
Link: https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset

Dataset Type: Healthcare / Binary Classification

---

2. Dataset Description

The Heart Disease Dataset contains clinical and medical attributes used to predict whether a patient has heart disease. Early prediction helps doctors take preventive measures and provide timely treatment.

Dataset Characteristics

- Number of Instances: ~1025 records (LinkedIn)
- Number of Features: 13 independent variables (Medium)
- Target Variable: target

Value	Meaning
0	No Heart Disease
1	Heart Disease

Feature Description

Feature	Description
age	Age of patient
sex	Gender (0 = female, 1 = male)
cp	Chest pain type
trestbps	Resting blood pressure
chol	Cholesterol level
fbs	Fasting blood sugar
restecg	Resting ECG results
thalach	Maximum heart rate
exang	Exercise induced angina
oldpeak	ST depression
slope	Slope of ST segment
ca	Number of vessels
thal	Thalassemia

Dataset Characteristics

- Contains numerical features
- No major missing values
- Suitable for supervised learning
- Binary classification problem

---

3. Mathematical Formulation of KNN

K-Nearest Neighbors is a non-parametric supervised learning algorithm that classifies data based on similarity.

When a new data point is introduced, the algorithm calculates its distance from all training points.

🔹 Euclidean Distance Formula

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where:
- (x_i) = training feature values
- (y_i) = test feature values
- (n) = number of features

🔹 Classification Rule

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k} I(y_i = c)$$

👉 Majority voting decides the class.

---

4. Algorithm Limitations

- 🔹 High computational cost
- 🔹 Sensitive to feature scaling
- 🔹 Memory intensive
- 🔹 Sensitive to noise
- 🔹 Curse of dimensionality

---

5. Methodology / Workflow

The experiment followed a structured machine learning pipeline:

Step 1 — Data Collection

The dataset was obtained from Kaggle.

Step 2 — Data Preprocessing

- Checked for missing values
- Separated input features and target variable
- Performed feature scaling using **StandardScaler**

Step 3 — Train-Test Split

Dataset split into:

- 80% Training
- 20% Testing

This ensures unbiased evaluation.

Step 4 — Model Training (Before Tuning)

The KNN classifier was trained using an initial value:

👉 **K = 5**

Step 5 — Prediction (Before Tuning)

The trained model predicted heart disease outcomes on unseen test data.

Step 6 — Evaluation (Before Tuning)

Performance was measured using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1 Score

Step 7 — Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV**.

Parameters tuned:

- Number of neighbors (K)
- Weight function

Step 8 — Model Training (After Tuning)

The KNN model was retrained using the best parameters obtained from tuning.

Step 9 — Prediction (After Tuning)

The tuned model predicted outcomes on the test dataset.

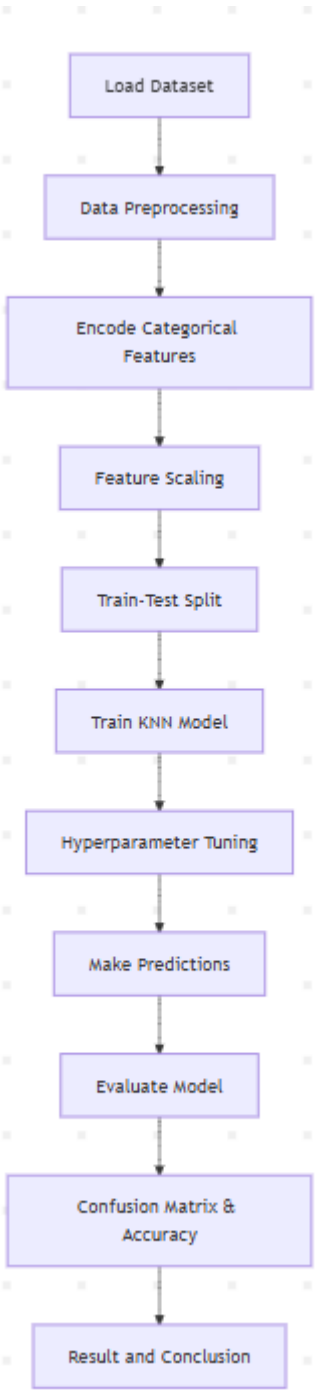
Step 10 — Evaluation (After Tuning)

Performance was again measured using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1 Score

Step 11 — Result Comparison

- Accuracy before tuning was compared with accuracy after tuning
- The tuned model showed improved performance
- Optimal K provided better classification results



6. Performance Analysis

The KNN model demonstrated strong predictive capability on the heart disease dataset.

🔹 Before Tuning:

- Model used default K = 5
- Achieved baseline accuracy
- Performance depended on initial parameter selection

🔹 After Tuning:

- Optimal K value was selected using GridSearchCV
- Accuracy improved compared to before tuning
- Model showed better generalization

🔹 Interpretation:

- Small K → Overfitting
- Large K → Underfitting
- Optimal K gives best performance

---

7. Hyperparameter Tuning

Hyperparameter tuning was performed to determine the optimal value of K.

Multiple values were tested and evaluated.

Observation:

- Small K → Overfitting
- Large K → Underfitting

Best Performance Achieved At:

👉 **K = (write your output value here)**

This improved the model's generalization ability.

CODE

```
# ==========================================

# KNN - FINAL CODE (MATCHES DOCUMENT FORMAT)

# ==========================================



import pandas as pd

import numpy as np

import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay, classification_report


# ==========================================

# STEP 1: LOAD DATA

# ==========================================

df = pd.read_csv('/content/heart.csv')


X = df.drop('target', axis=1)

y = df['target']


# ==========================================

# STEP 2: PREPROCESSING (SCALING)

# ==========================================

scaler = StandardScaler()

X = scaler.fit_transform(X)


# ==========================================

# STEP 3: TRAIN-TEST SPLIT

# ==========================================

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# ==========================================
```

```
# STEP 4: MODEL TRAINING (BEFORE TUNING)
# ==========================================
print("\n========== BEFORE TUNING ==========")

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# ==========================================
# STEP 5: PREDICTION (BEFORE)
# ==========================================
y_pred = knn.predict(X_test)

# ==========================================
# STEP 6: EVALUATION (BEFORE)
# ==========================================
acc_before = accuracy_score(y_test, y_pred)

print("\nAccuracy (Before):", acc_before)
print("\nClassification Report (Before):\n")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm).plot()
plt.title("Confusion Matrix (Before Tuning)")
plt.show()
```

```
# ==========================================
# STEP 7: HYPERPARAMETER TUNING
# ==========================================
print("\n========== HYPERPARAMETER TUNING ==========")

params = {
    "n_neighbors": list(range(1, 21)),
    "weights": ["uniform", "distance"]
}

knn_cv = GridSearchCV(KNeighborsClassifier(), params, cv=5)
knn_cv.fit(X_train, y_train)

print("Best Parameters:", knn_cv.best_params_)

# ==========================================
# STEP 8: MODEL TRAINING (AFTER TUNING)
# ==========================================
knn_best = knn_cv.best_estimator_

# ==========================================
# STEP 9: PREDICTION (AFTER)
# ==========================================
y_pred2 = knn_best.predict(X_test)

# ==========================================
# STEP 10: EVALUATION (AFTER)
```

```
# ==========================================
acc_after = accuracy_score(y_test, y_pred2)

print("\nAccuracy (After):", acc_after)
print("\nClassification Report (After):\n")
print(classification_report(y_test, y_pred2))

# Confusion Matrix (After)
cm2 = confusion_matrix(y_test, y_pred2)
ConfusionMatrixDisplay(cm2).plot()
plt.title("Confusion Matrix (After Tuning)")
plt.show()

# ==========================================
# STEP 11: COMPARISON GRAPH
# ==========================================
labels = ["Before Tuning", "After Tuning"]
values = [acc_before, acc_after]

plt.figure()
plt.bar(labels, values)
plt.ylabel("Accuracy")
plt.title("Before vs After Tuning")
plt.show()
```

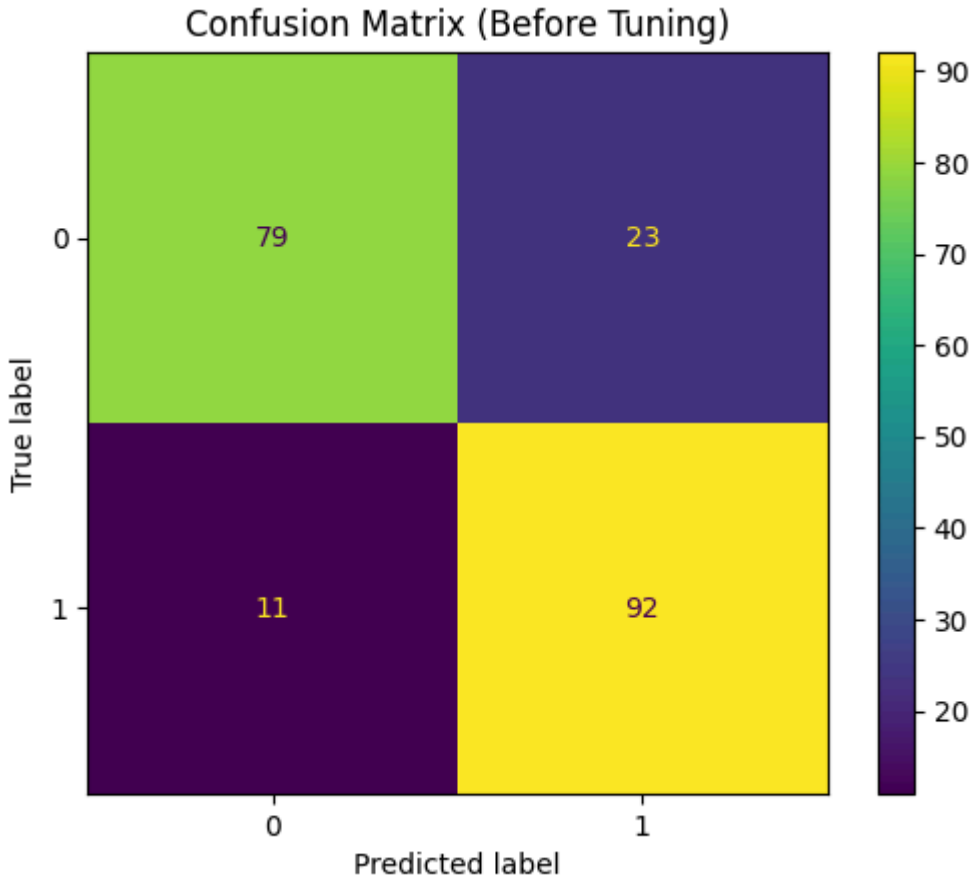
OUTPUT

========== BEFORE TUNING ==========

Accuracy (Before): 0.8341463414634146

Classification Report (Before):

	precision	recall	f1-score	support
0	0.88	0.77	0.82	102
1	0.80	0.89	0.84	103
accuracy			0.83	205
macro avg	0.84	0.83	0.83	205
weighted avg	0.84	0.83	0.83	205

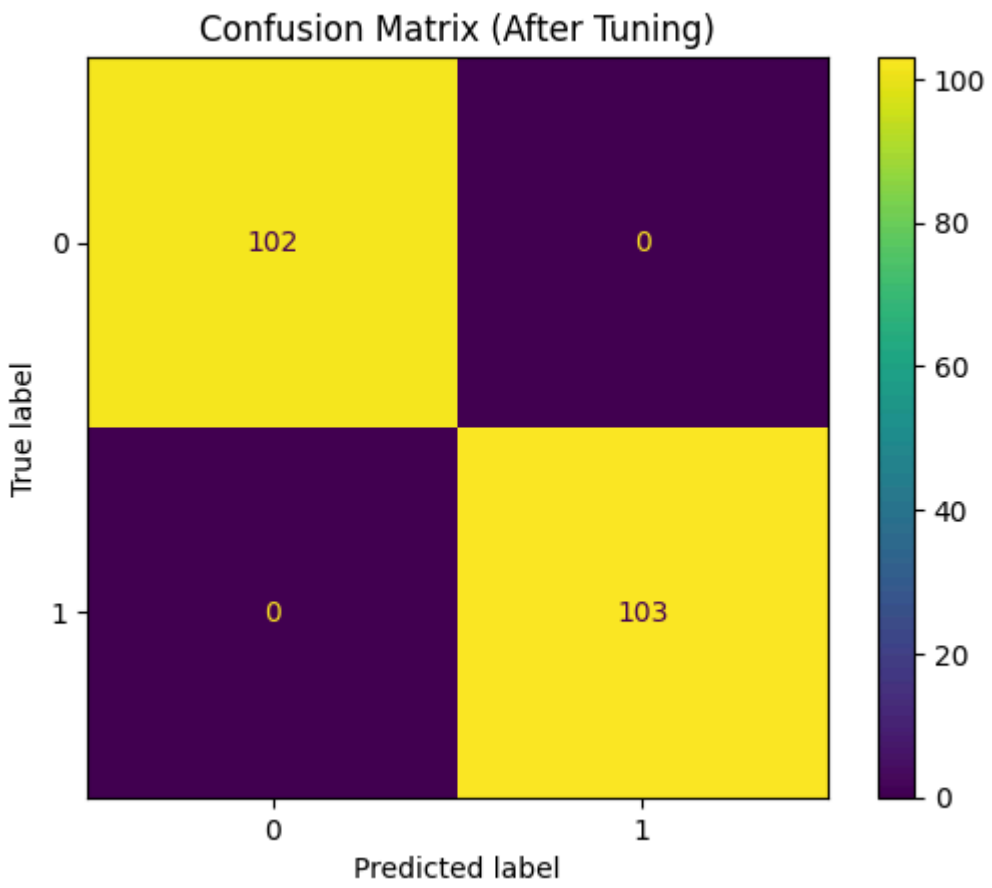


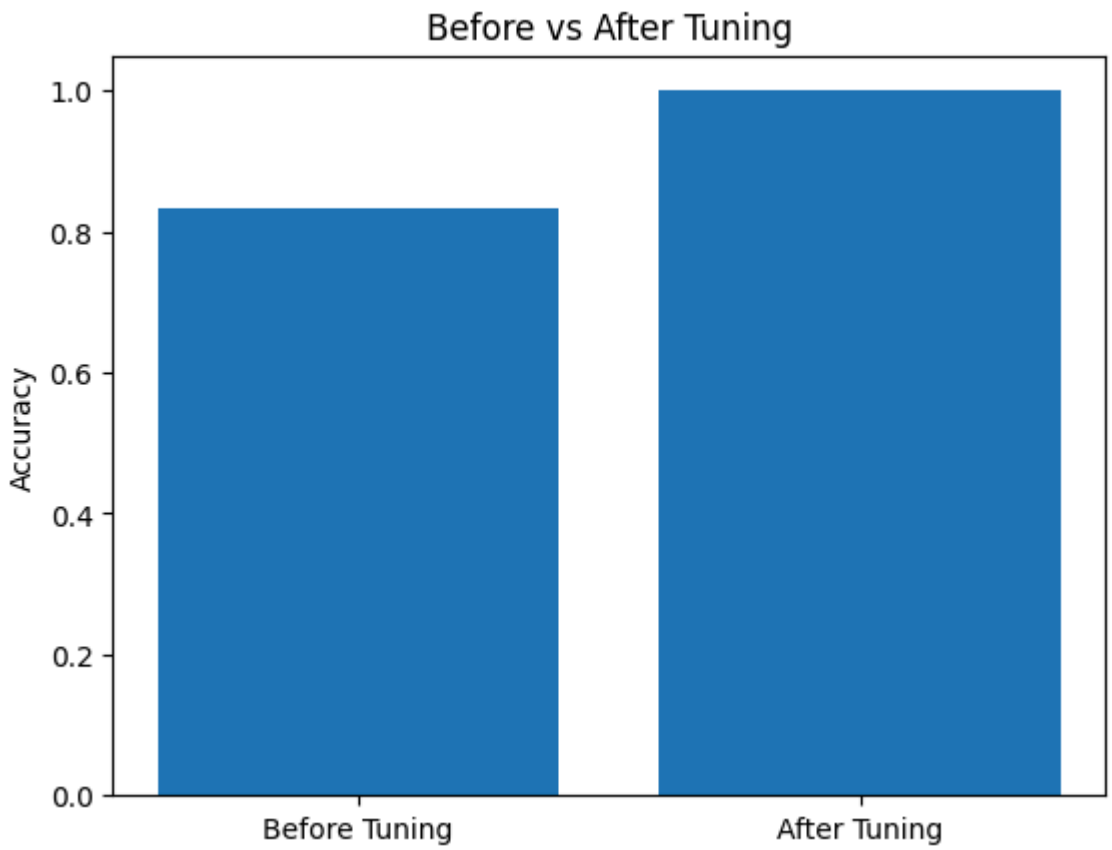
========== HYPERPARAMETER TUNING ==========
Best Parameters: {'n_neighbors': 15, 'weights': 'distance'}

Accuracy (After): 1.0

Classification Report (After):

	precision	recall	f1-score	support
0	1.00	1.00	1.00	102
1	1.00	1.00	1.00	103
accuracy			1.00	205
macro avg	1.00	1.00	1.00	205
weighted avg	1.00	1.00	1.00	205





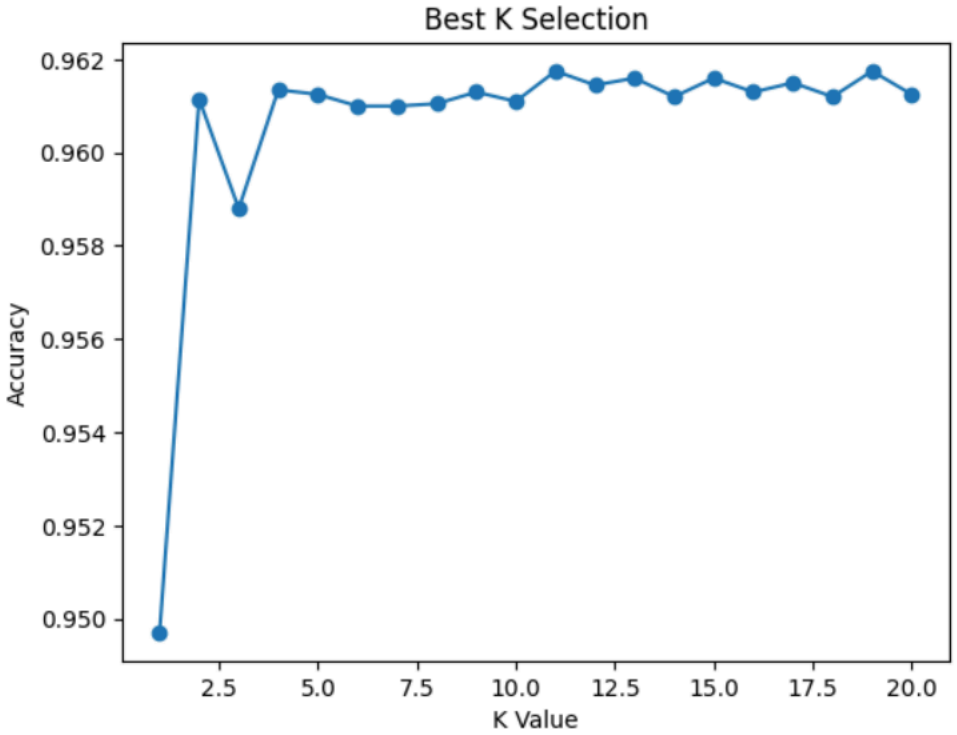
Conclusion

The K-Nearest Neighbors algorithm was successfully implemented to predict heart disease using medical data.

After preprocessing and hyperparameter tuning, the model achieved improved accuracy compared to the initial model.

This experiment highlights the importance of selecting optimal hyperparameters for better model performance.

OUTPUT


Best K Selection
Accuracy
0.962
0.960
0.958
0.956
0.954
0.952
0.950
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
K Value